The **grammar** and **parser** commands

Basil Fürer Dmitriy Traytel

November 7, 2026

Abstract

A definitional package for indentation-sensitive, context-free grammars in Isabelle/HOL. **grammar** elaborates a grammar specification with semantic actions into a grammar constant, per-nonterminal decoders and a mutual inductive relation. **parser** emits an executable parser for it together with soundness and completeness theorems. The parser generator backends are extensible; this package provides an LALR(1)-style backend with layout-aware lookahead out of the box.

Contents

1 Overview **2**

2 The span of a yield **2**

3 Grammar specifications **3**
3.1 Meaning of the annotations 8
3.2 Precedence 8
3.3 Composition by reference 9
3.4 Templates and operators 10

4 Parsers **11**

5 What is defined **12**
5.1 Constants and facts of a grammar 12
5.2 Constants of a parser 13

6 Theorems **13**

7 Diagnostics **14**

8 Backend: the LR parser **17**

9 Example **18**

1 Overview

The package defines grammars and parsers for indentation-sensitive context-free grammars in the sense of Adams [1]. A grammar is a value of type (*'n*, *'tok*) *is-cfg* (`../Indentation_Sensitive.thy`): a list of productions over non-terminals *'n* and token classes *'tok*, in which every right-hand-side symbol carries an indentation relation *irel*. Parser input is a list of token records (*'tok*, *'ch*, *'m*) *token-scheme* (`../Token.thy`); the token class *tok* and the column *s-col* of the *span* are all the parser reads, the remaining fields and any record extension *'m* pass through to the semantic actions.

Two commands make this usable from Isar. **grammar** turns a grammar specification with semantic actions into a grammar constant and per non-terminal decoders, optionally also into an inductive relation between words and values. **parser** emits an executable parser[1] for such a grammar together with soundness and completeness theorems.

A grammar may use the nonterminals of an earlier grammar, as *a.X*, and parametric nonterminals (templates) such as *Sep1*(*X*, *S*), with the postfix operators $*$ $+$ *?* for lists and options. Both are expanded before anything is defined, so the result is an ordinary closed grammar. A rejected word gets a located error report, with hand-written messages if given, and can be recovered into a tree; that side is unverified and cannot change what the parser accepts.

The parser generator is extensible; this package provides one backend, *LR*: an LALR(1)-style parser with layout constraints, running the paper's exact $k = 0$ machine under one token of lookahead propagated over the LR(0) automaton. Its determinism premise of completeness is decided by a static check (**by** *eval*).

2 The span of a yield

An alternative that writes @*span* passes its action the span of its yield, from the first token to the last (*word-span*): line, column and offset of the first token, the length up to the end of the last, and the concatenated texts of the tokens. The decoders compute it on the children of a node (*yield-span*), the relation on the word; *ast-yield-erase* makes the two agree.

fun *ast-yield* :: ‹('*m*, '*ch*, '*n*, '*tok*) *ast* ⇒ ('*m*, '*ch*, '*tok*) *tword*› **where**
  ‹*ast-yield* (*Leaf t*) = [*t*]›
| ‹*ast-yield* (*Node n ts*) = *concat* (*map ast-yield ts*)›

definition *word-span* :: ‹('*m*, '*ch*, '*tok*) *tword* ⇒ '*ch span*› **where**
  ‹*word-span w* =

[1] Semantic actions must be executable if the parser is to be.

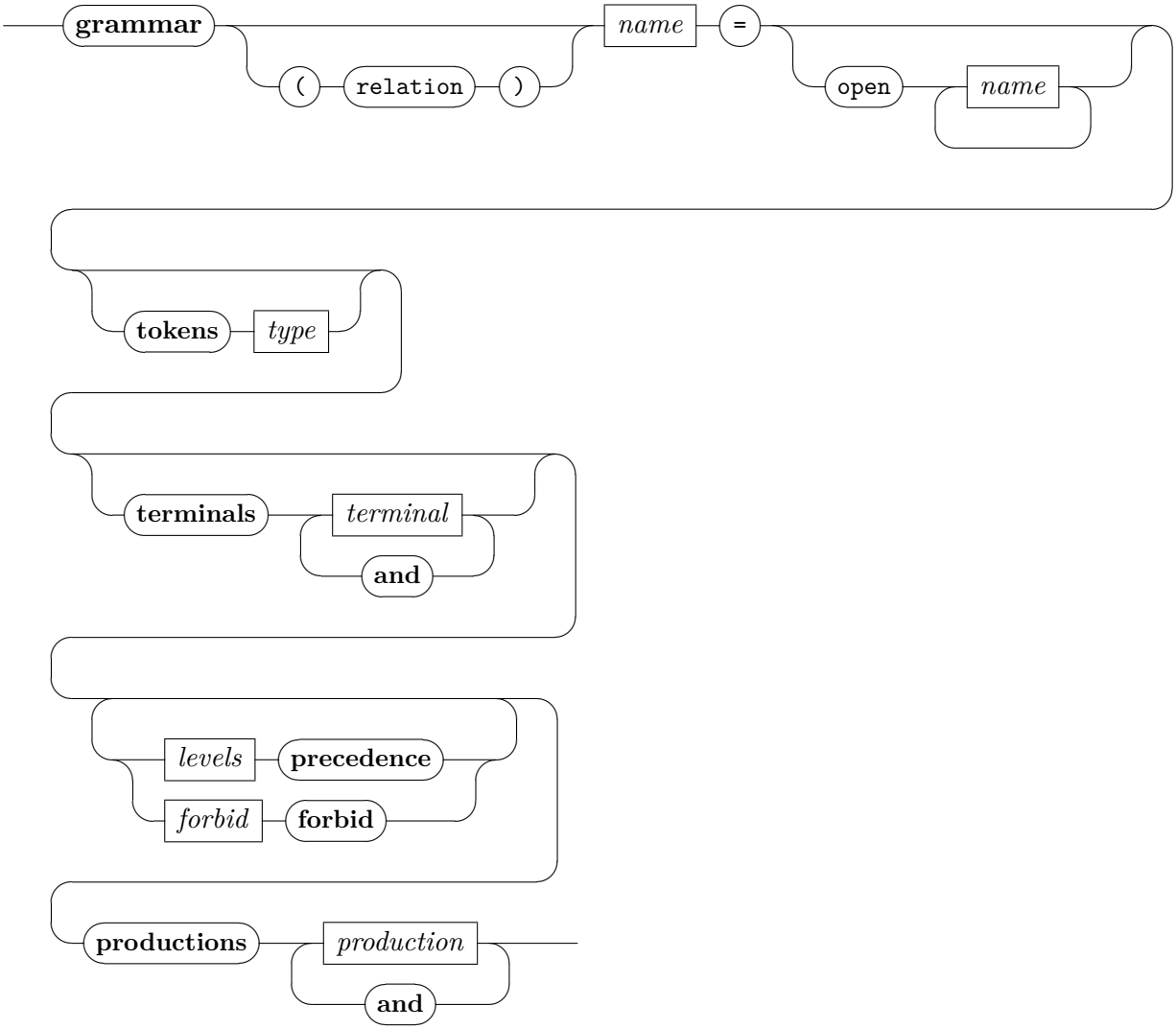
```
  (case w of
    [] ⇒ ⦇s-line = 0, s-col = 0, s-off = 0, s-len = 0, s-txt = []⦈
  | t # - ⇒
      ⦇s-line = s-line (span t), s-col = s-col (span t), s-off = s-off (span t),
       s-len = s-off (span (last w)) + s-len (span (last w)) − s-off (span t),
       s-txt = concat (map (s-txt ∘ span) w)⦈)›

definition yield-span :: ‹('m, 'ch, 'n, 'tok) ast list ⇒ 'ch span› where
  ‹yield-span xs = word-span (concat (map ast-yield xs))›

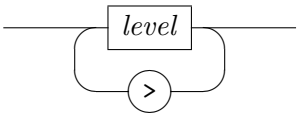
lemma ast-yield-erase [simp]: ‹ast-yield (erase pt) = pyield pt›
  by (induction pt) (simp-all cong: map-cong)
```

3 Grammar specifications

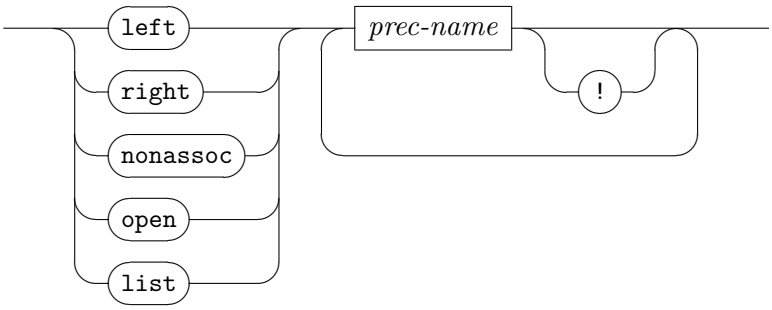
grammar : *local-theory* → *local-theory*


grammar
(
relation
)
name
=
open
name
tokens
type
terminals
terminal
and
levels
precedence
forbid
forbid
productions
production
and

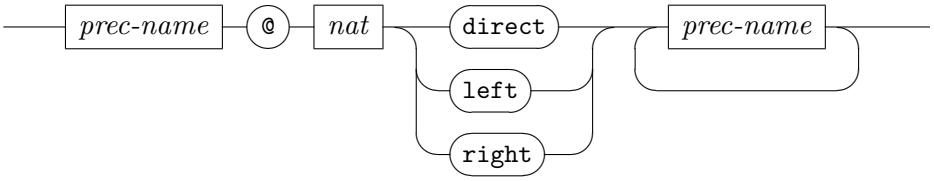
levels


level
>

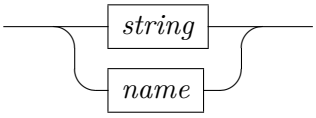
level



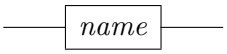
forbid



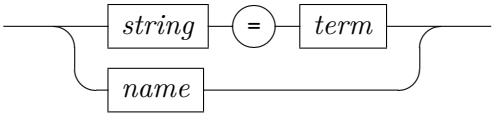
prec-name



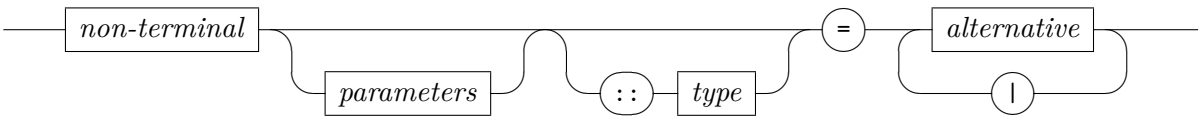
non-terminal



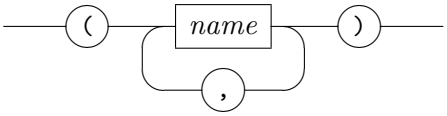
terminal



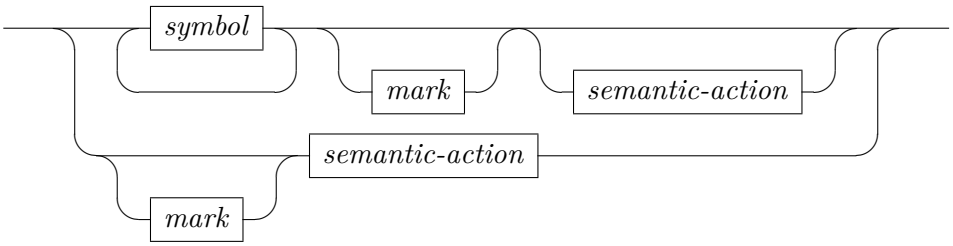
production



parameters



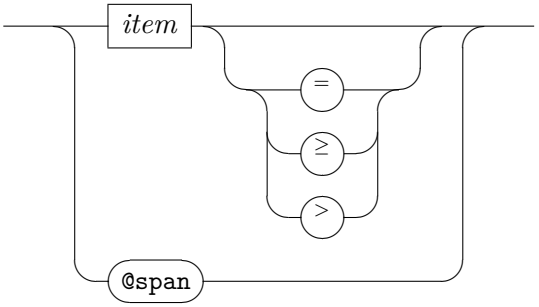
alternative



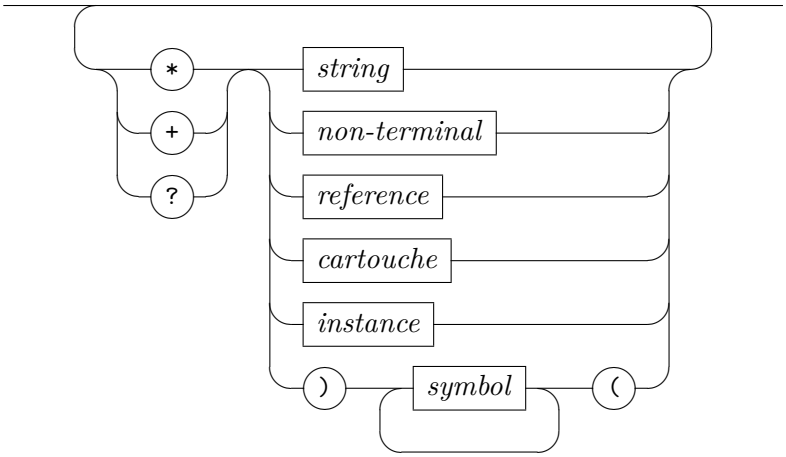
mark



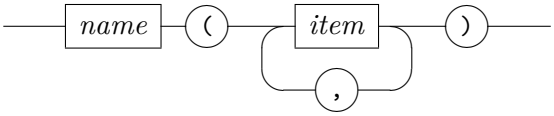
symbol



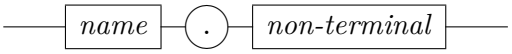
item



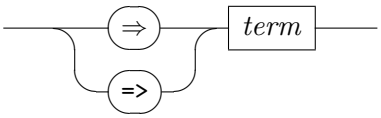
instance



reference



semantic-action



The categories *name*, *string*, *type*, *term* and *cartouche* are those of the Isar reference manual. The annotations may also be written in ASCII: `^=`, `^>=` and `^>`. Templates, instances and the operators are described below.

grammar $G = \ldots$ checks the specification and defines the grammar constant G (**grammar** (*relation*) $G = \ldots$ also defines the relation family, see below).

The first non-terminal is the start symbol.

tokens T fixes the token record type. T is a ground instance of ($'tok$, $'ch$, $'m$) *token-scheme*: the plain record ($'tok$, $'ch$) *token*, or a client extension **record** *my-token* = (*my-tok*, *char*) *token* + ... (see below).

A type that is not a token record abbreviates (T, *unit*) *token*; without a **tokens** clause the token type is *string*.

terminals declares the terminal symbols. A *fixed spelling* $s = t$ lets the quoted spelling s stand for the token $t :: 'tok$ in the productions without binding it in semantic actions; a bare name declares a *token class*, used by its name in the productions with the whole token record as its value. A bare name that is neither a non-terminal nor declared is read as a term of type $'tok$ (for the *string* token type: the string itself).

productions lists the non-terminals. A group $A :: \tau = alt_1 \mid \ldots \mid alt_n$ declares the non-terminal A with *carrier* τ (the type of its semantic values and its alternatives). The ascription can be omitted when the carrier follows from the actions.

An alternative is a sequence of symbols, each with an optional annotation, followed by an optional semantic action. A symbol is a non-terminal, a fixed terminal, or an inline token $\langle t \rangle$ with $t :: 'tok$.

A non-terminal B contributes its value of type τ_B, and a token the complete token record.

The semantic action $\Rightarrow f$ computes the value of the alternative from the values of its value binding symbols, in order: $\tau_1 \Rightarrow \ldots \Rightarrow \tau_k \Rightarrow \tau_A$.

An alternative without action must have exactly one value symbol, whose value is passed on.

@*span* is a value of type $'ch$ *span*, in the place where it is written among the value symbols: the span of the alternative's yield, from its first token to its last (*word-span*, section 2). It is not a grammar symbol; the grammar

constant, the tables and the diagnostics do not see it.

An alternative with @*span* needs a symbol that derives no empty word, or it has no first token. @*span* takes no annotation, is written once per alternative, and stands at the top level of an alternative, not inside an operator's group or as a template argument (a template alternative may have it).

The annotation of a symbol is its indentation relation: $^{=}$ for *Eq*, $^{\geq}$ for *Ge*, $^{>}$ for *Gt*; without annotation it is *Any*.

3.1 Meaning of the annotations

The indentation of a terminal is the column of its token, a nonterminal has no token of its own and its indentation is a free choice restricted only by the annotations.

A sentential form annotates every symbol with an indentation (column). Expanding $(A,\ i)$ by a production $A = {X_1}^{r1}\ \ldots\ {X_n}^{rn}$ chooses for each symbol an indentation j_k with *irel-sat* $r_k\ j_k\ i$:

$$\begin{aligned} \textit{irel-sat Eq } j\ i &\longleftrightarrow j = i \\ \textit{irel-sat Gt } j\ i &\longleftrightarrow i < j \\ \textit{irel-sat Ge } j\ i &\longleftrightarrow i \leq j \\ \textit{irel-sat Any } j\ i &\longleftrightarrow \textit{True} \end{aligned}$$

So $^{=}$ aligns a symbol with the nonterminal being expanded, $^{>}$ indents it further, $^{\geq}$ allows both, and no annotation places no constraint. A block whose members are aligned is written $\textit{Block} = \textit{Stmt}^{=} \mid \textit{Block}^{=}\ \textit{Stmt}^{=}$: all statements share the indentation of the block. A grammar without annotations is an ordinary context-free grammar.

ilang G is the set of token words derivable in this way.

3.2 Precedence

precedence right $\hat{}$ $>$ *left* $*$ */* $>$ *left* $+$ $-$ $>$ *nonassoc* $==$ declares levels, $>$ binding tighter. A production goes by its (*prec NAME*), else by its last terminal, written as its spelling or its constructor name; a level's name need not be a terminal, as in $-\ E$ (*prec UMINUS*). Several lines merge: a name in two lines is one level, with one associativity, and the order is the transitive closure of the lines. A cycle is an error. Two levels no line relates stay ambiguous, and the parser reports the conflict.

left, *right* and *nonassoc* are the usual readings. *open* leaves the left operand of its productions unconstrained, for forms that end in an open construct. *list* does not constrain the productions of the level against each other on one nonterminal, for flat chains. A prefix form on the right of a tighter operator is normally allowed, as in $a * -\ b$; ! on an item forbids that. *forbid* p @ k

right c says what no level says: c is not on the right spine of child k of p (*left*: the left spine, *direct*: the child itself).

The declarations belong to the grammar, the restriction to the parser: each parser of a declaring grammar defines *p-prec* and *p-gram* = *restricted* G *p-prec* and is built from *p-gram*, from its own start; a grammar without declarations is not touched. Such a theory imports `LR_Parser.LR_Prec_Parse`. *p.sound* and *p.complete* are stated against G and *prec-ok*, on trees of G; *p.sound-lang* and *p.complete-lang* against *prec-lang*. With **grammar** (*relation*) the relation family is defined as usual but does not read the declarations, so no theorem is stated against it, and **parser** says so.

3.3 Composition by reference

A symbol $a.X$, with a an earlier grammar and X one of its nonterminals, uses X in another grammar b. The productions of a reachable from X are copied into b, after b's own, renaming each nonterminal Y of a to $a.Y$; a nonterminal that a copied itself, $c.Z$, keeps its name, so one reached along two paths is copied once. b is then a closed grammar like any other, and $a.X$ is one of its nonterminals, also as the start of a **parser** entry. In constant and fact names the dot is an underscore: *b--dec-a-X*, *b-a-X*.

The token types of a and b must be the same. The fixed spellings of the copied productions' terminals come with them and may be used in b; a spelling that stands for two different tokens is an error. The precedence declarations of a apply to the copied productions. A defined nonterminal has a plain name, and two nonterminals with one name in the constants (*a-X* and $a.X$) are an error.

If a and b both have a relation (**grammar** (*relation*)) and agree in being indentation-sensitive or not, *b.a-X-iff*: *b-a-X* i w x $\longleftrightarrow$ *a-X* i w x links the two for each $a.X$ that b names, so what a parser of b proves about $a.X$ is a fact about a. The decoders are defined anew for the copy.

grammar b = **open** a lets b write a nonterminal X of a without the qualifier: a bare name that is not a nonterminal or terminal class of b itself, nor a template parameter, is $a.X$ if a defines X. So b's own names come first, a name that two opened grammars define must be qualified, and a name no opened grammar defines is read as before (a token, under a token type other than *string*). A template P of an opened grammar comes before one of the same name elsewhere. Only the names change: X under *open* a is the copy $a.X$, with the same constants.

Messages name a copy without the qualifier ($a.Atom$ as *Atom*, $a.Block(Bind)$ as *Block*(*Bind*)), except where two nonterminals would then read alike.

Under **declare** [[*grammar-nat-labels*]] a grammar's nonterminals are numbers in HOL, (*nat*, *'tok*) *is-cfg*, instead of their names: *0* for the first, then *2*, *3*, ... in the order of declaration, the copies after the grammar's own

(*1* is skipped, as the simplifier rewrites it to *Suc 0*). The constants, relations, decoders and parsers are the same otherwise, and messages still show the names. The generated proofs then compare numerals instead of strings, which pays for long qualified names such as *caraml-exprs.Atom*. A grammar with *precedence* of its own, or that copies nonterminals of one with *precedence*, keeps its names, since the restriction of `../lr/LR_Restrict.thy` works on strings.

3.4 Templates and operators

A production group with parameters, $P(X_1, \ldots, X_n) :: \tau = \ldots$, is a *template*. It defines nothing by itself. Each use $P(a_1, \ldots, a_n)$, with the parenthesis written directly after the name, is an *instance*: the group with every X_i replaced by a_i, an ordinary nonterminal named $P(a_1, \ldots, a_n)$, made once per distinct argument list. An argument is a symbol (a nonterminal, a terminal, an instance, a reference $a.X$), a symbol with an operator, or a group of symbols, which then stands in place of the parameter as a sequence. The actions of the template see the right-hand side after the replacement: a parameter bound to a fixed terminal binds nothing. Type variables in the ascription of a template are inferred per instance. A template of an earlier grammar g is used as $g.P(\ldots)$ or, when no other template has the name, $P(\ldots)$; its nonterminals then come in as $g.Y$, its spellings with them. A grammar of templates only defines nothing.

The postfix operators over a symbol or a group $(s_1 \ \ldots \ s_n)$ generate a nonterminal: $s*$ and $s+$ are left-recursive lists of carrier *'x list*, $L = \varepsilon \mid L\ s_1 \ \ldots\ s_n$ (and $L = s_1 \ \ldots \ s_n \mid \ldots$) with xs @ $[x]$, and $s?$ is an option, *None* or *Some x*. The element x is the value of the one value symbol of the group, or the tuple of several. A group without value symbol, as in $;?$, generates a nonterminal that binds nothing at its use. The recursive occurrence L carries the annotation of the group's first symbol, so $(Stmt^{=})+$ is $L = Stmt^{=} \mid L^{=}\ Stmt^{=}$, and a grammar without annotations stays one. An annotation after the operator, $(;\ X)*^{=}$, is that of the generated nonterminal at its use. A group without operator is inlined and takes no annotation.

Generated nonterminals are named as written, *Block*(*Decl*), (; *Decl*)∗ (a spelling without its quotes, which a HOL string cannot hold); in constant names they are identifiers: *Block-Decl*, *Decl-list*, *Decl-list1*, *Decl-opt* for an operator over one nonterminal, and *A-1*, *A-2*, ... for the groups of the alternatives of *A*. Messages show a list or option by its element: *Decl*+, *Decl*∗, *Decl?* and the prelude's *List1*(*Decl*) or *Sep1*(*Decl*, ;) as *a Decl*, a group as its first symbol ((; *Decl*)∗ as *';'*); other instances, as *Block*(*Decl*), keep their names.

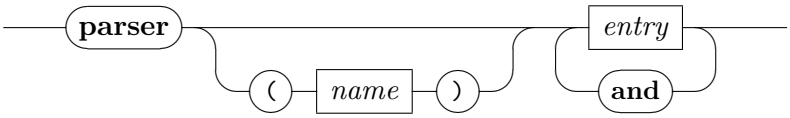
grammar *Grammar-Prelude* below provides the generic templates.

grammar *Grammar-Prelude* =

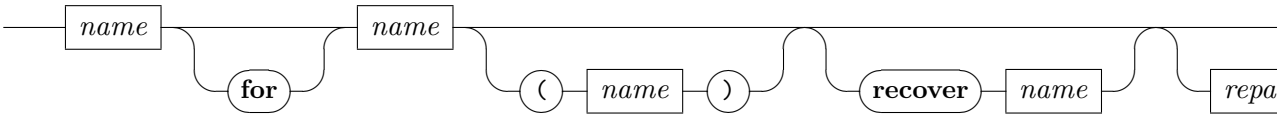
productions
List(*X*) :: ‹*'x list*› = ⇒ ‹[]› | *List*(*X*) *X* ⇒ ‹λ*xs x*. *xs* @ [*x*]›
and
List1(*X*) :: ‹*'x list*› = *X* ⇒ ‹λ*x*. [*x*]› | *List1*(*X*) *X* ⇒ ‹λ*xs x*. *xs* @ [*x*]›
and
Opt(*X*) :: ‹*'x option*› = ⇒ ‹*None*› | *X* ⇒ ‹*Some*›
and
Sep1(*X*, *S*) :: ‹*'x list*› = *X* ⇒ ‹λ*x*. [*x*]› | *Sep1*(*X*, *S*) *S X* ⇒ ‹λ*xs x*. *xs* @ [*x*]›
and
Sep(*X*, *S*) :: ‹*'x list*› = ⇒ ‹[]› | *Sep1*(*X*, *S*)

4 Parsers

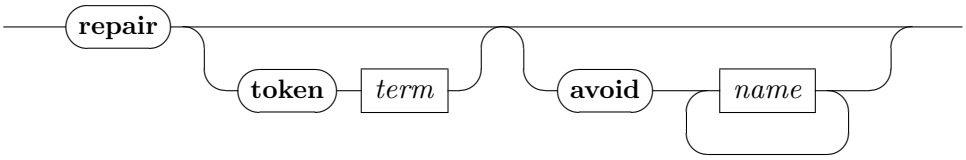
parser : *local-theory* → *local-theory*



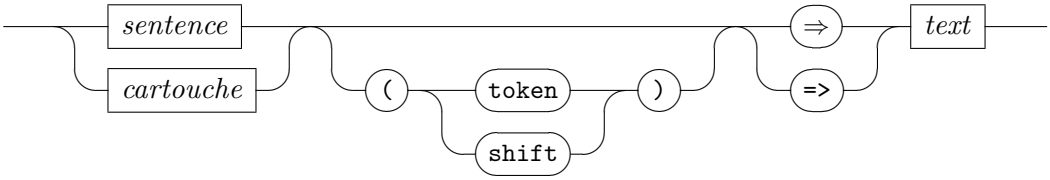
entry



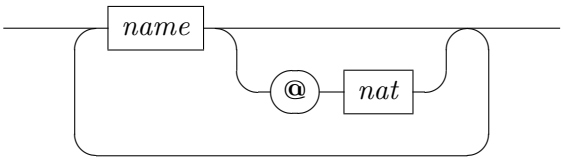
repair



message



sentence



parser *p* **for** *G* defines *p* :: *word* ⇒ τ_S *option* for the start symbol *S* of *G*, together with soundness and completeness theorems. The optional first name selects the parser generator (by default *LR*) for all entries of the command.

parser *p* **for** *G* (*A*) parses from the nonterminal *A* of *G* instead: *p* :: *word* ⇒ τ_A *option*, built from *G*(|*istart* := ″*A*″|), the grammar with the same productions and start *A*. Its theorems are those of the start symbol with *A* for *S*: the relational forms are about *G-A 0 w x* and the language forms about *ilang* (*G*(|*istart* := ″*A*″|)), the language of *A*. (*S*) with *S* the start symbol is the plain form.

parser (*ast*) *p* **for** *G* defines *p* :: *word* ⇒ ('*m*, '*ch*, *string*, '*tok*) *ast option*, which returns the tree of the word and evaluates no action: the decoder is *Some*. For an outline or folding in an editor, or a grammar whose actions are not written yet (they are still checked by **grammar**). The theorems speak of the tree, see below; the option combines with the others, with entries and with the diagnostics, whose *p-diag* and *p-recover* then return trees as well.

parser *p* **for** *G* **and** *q* **for** *G* (*A*) declares several parsers for one grammar, one after the other (parsers for another grammar need a command of their own). The entries share the grammar's decoders, its relation and the decoder and relation lemmas the theorems are discharged with, proved once for the command; each entry has its own tables, from its own start.

The clauses after the start symbol, and the option (*no-diagnostics*), are about rejected words; they are described with the diagnostics below.

5 What is defined

5.1 Constants and facts of a grammar

For **grammar** *G* with non-terminals *A* of carrier τ_A and start symbol *S*:

The grammar constant *G* :: (*string*, '*tok*) *is-cfg* with its equation *G-def*. Nonterminals are represented by their names, the start symbol has indentation *0*.

One decoder *G--dec-A* :: ('*m*, '*ch*, *string*, '*tok*) *ast* ⇒ τ_A *option* per nonterminal, with the equations *G--dec-A.simps*. It evaluates the actions over a parse tree, matching each node against the productions of *A*. The decoders

are defined together through *ast-rec*.

With (*relation*): the **inductive** family *G-A* :: *nat* ⇒ *word* ⇒ τ_A ⇒ *bool*, where *G-A i w x* holds if the word *w* derives from (*A*, *i*) with value *x*, there is one introduction rule per alternative. The predicates are defined by one (mutual) inductive definition per strongly connected component of the nonterminals, lower components first. The facts are *G.intros*, *G.inducts* (the induction rule of each predicate, over its component) and *G.lang*: (∃ *x*. *G-S 0 w x*) ⟷ *w* ∈ *ilang G*.

For a grammar without annotations the indentation argument is dropped: *G-A* :: *word* ⇒ τ_A ⇒ *bool* and *G.lang*: (∃ *x*. *G-S w x*) ⟷ *w* ∈ *ilang G*.

A copied nonterminal *a.X* of a composed grammar *b* gets the decoder *b--dec-a-X* and, with (*relation*), the predicate *b-a-X* like any other; a template defines nothing, its instances are ordinary nonterminals.

5.2 Constants of a parser

The definition of *p* is *p w* = *Option.bind* (*dparse G Shn w*) *G--dec-S* (for an entry (*A*), *dparse* (*G*⦇*istart* := ″*A*″⦈) *Shn w* and *G--dec-A*): *dparse* is the verified driver of `../lr/LR_Driver.thy`, an LALR(1)-style parser over the LR(0) automaton whose lookahead cells carry the layout constraints, *Shn* a fresh name for the augmented start symbol, and *G--dec-S* the decoder of the start symbol.

A parser of a grammar with *precedence* declarations also defines *p-prec* and *p-gram* and is built from *p-gram*, see the precedence subsection above. The constants of the diagnostics (*p-diag*, *p-messages*, *p-recover*, *p-coverage* and the others) are described with them below, the tables of the code with the backend.

6 Theorems

The facts *p.sound* and *p.complete* relate the parser to the language:

p.sound: *p w* = *Some x* ⟹ *w* ∈ *ilang G*
p.complete: *w* ∈ *ilang G* ⟹ ∃ *x*. *p w* = *Some x*

For a grammar declared with (*relation*), *p.sound* and *p.complete* relate it to the relation of the start symbol:

p.sound: *p w* = *Some x* ⟹ *G-S 0 w x*
p.complete: *G-S 0 w x* ⟹ *p w* = *Some x*

The language forms are then available as *p.sound-lang* and *p.complete-lang*; should the bridge to the relation fail, the command warns and keeps the language forms under the plain names.

With (*ast*), *p.sound* and *p.complete* are stated against the parse trees of

G, with or without a relation:

$$p.sound\text{: } p\ w = Some\ t \Longrightarrow ast\text{-}of\ G\ (Nt\ (istart\ G))\ (istart\text{-}ind\ G)\ t\ w$$
$$p.complete\text{: } wf\text{-}ptree\ G\ pt \Longrightarrow proot\ pt = (Nt\ (istart\ G),\ istart\text{-}ind\ G) \Longrightarrow p\ (pyield\ pt) = Some\ (erase\ pt)$$

that is, t is the erasure of a parse tree of w, and every parse tree of a word erases to the tree p returns, which is therefore unique; *p.sound-lang* and *p.complete-lang* are the language forms, with no premise on a decoder.

The generic theorems behind these facts carry premises that the command discharges when p is defined. Soundness needs only *fresh-name*, proved by simplification, and is therefore always unconditional.

Completeness needs in addition:

mdet G Shn, determinism of the driver on the grammar. The command evaluates the static report *rlalr-conflicts G Shn*, the LALR conflict analysis extended with the layout constraints. If it is empty, *mdet* follows by *rlalr-sufficient* and the command says so; otherwise it warns, lists the conflicting states, and keeps *mdet* as a premise of *p.complete*. The check is sufficient, not necessary: an ambiguous grammar always fails it, a deterministic one may fail it too. The parser is emitted either way; a run of it that revisits a configuration without consuming input (a unit self-loop whose reduce is licensed at the end of the input, say) ends with *None*.

Totality of the decoders on the parse trees of G.

For the relational forms, *acc-conflicts G Shn* = [] (again by evaluation).

Premises that cannot be discharged stay on the theorem as assumptions, so **thm** *p.complete* shows what has been established.

For a grammar with precedence declarations the theorems are stated against *prec-ok*, and a composed grammar adds the link theorems *b.a-X-iff*; both are described with the grammar specifications above.

7 Diagnostics

The parser p says only *None* on a rejected word. Each parser also has a diagnostics side, unless **parser** (*no-diagnostics*) turns it off (the option *diagnostics* states the default; p and its tables are the same either way): *p-diag w* is *Inr x* exactly when $p\ w = Some\ x$, and otherwise *Inl e* with e a record *perr* (`../lr/LR_Diag.thy`): the index and token at which the word failed (*None* at the end of the input), the state, the kind (no action, a cycle of reductions, a token offside or misaligned), the expected terminals and, under layout, their columns, the anchor, the stack with its spans, and any repair. *perr-message* turns it into text, and the *errors* block overrides that text state by state. Two things in that text come from the grammar. A pair of terminals that balance in every production and of which the second ends

one (*brackets*) is a bracket: when the innermost one on the stack is the error, the message says "*'(' at 2:24 is not closed*" and the generic text becomes a note. And a long *expected*: list is shortened to the nonterminals that may start there, by the name they have in the grammar (a generated list by its element, see above), so a nonterminal's name is part of the messages.

parser (*diagnostics*) *p* **for** *G errors s* ⇒ ‹*m*› | ... attaches hand-written messages to the error states of *p*, by example (Jeffery 2003, Pottier 2016). The word *s* is written with the terminals' spellings or constructor names, each with an optional column after @; a token without one follows the previous token. For tokens with further fields, or to use a lexer, *s* is a cartouche with a term of the word type. The last token of *s*, or the end of *s*, is where it fails.

The command evaluates *p-probe*, the error state of a word without its decoder and expected set, once on all the sentences, with no theorem, which gives the constant *p-messages*, plain data that *p-render* consults: the text replaces the primary line of the default message, $*0* to $*9* standing for the text of the stack entries from the top. The key is the state where the error is detected, with (*token*) that state and the offending terminal, which goes first, and with (*shift*) the state after the last shift. A word that is accepted, a key taken by an earlier sentence, a terminal that does not exist, and a sentence that cannot be evaluated are warnings, and the sentence is ignored; a word rejected before its last token and a hole below the stack are warnings only. Since keys are computed at each elaboration, a grammar edit that renumbers the states needs no change of the block.

parser (*diagnostics*) *p* **for** *G recover e* adds *p-recover*, for a grammar where the terminal *e* (a spelling or a constructor) stands for broken input: an ordinary terminal with ordinary productions such as *Expr* = ... | *ERR* ⇒ ‹*EErr*›, which builds an error node. It is checked like any other terminal, so a conflict it takes part in is reported, and it is left out of the expected sets of the reports. *p-recover w* is (*errs*, *p w'*): $w' = w$ and *errs* = [] when *p* accepts *w*; otherwise *p-repair*, which is not verified, gives w', and *errs* are the reports at each stop, never empty, with *pe-index* into *w*. At a stop the stack is popped to a state that can shift *e*, the tokens are skipped up to one that continues, and the whole span becomes one *e* carrying the source span of what it replaces (Menhir's *error*, Bison's three tokens of quiet between reports); under layout it takes a column the state accepts. A word with no place for *e* gives *None* with its reports. The word w' is parsed by *p* itself, so *p-recover* costs nothing on good input and all the theorems of *p* apply to the tree. Two facts come with it: *p-recover.clean*, that no reports means the tree is the one of *p w*, and *p-recover.no-error*, that the tree of a word without *e* has no *e* in its yield. Nothing is proved about *p-repair*. Without *diagnostics*, or with a name that is not a terminal of *G*, *recover* is a warning and is ignored.

parser (*diagnostics*) *p* **for** *G repair* adds *p-recover* as well, for any grammar: at a stop, before *e* if there is a *recover*, it searches for a repair sequence as in CPCT+ (Diekmann and Tratt 2020): tokens inserted and deleted at the least cost, then kept, ranked by how far parsing goes on. The repair is in *pe-repair* of the report and in its message. A budget of nodes for the whole word bounds the search, as HOL has no clock (*rnodes*, in `../lr/LR_Recover.thy`), and compatible nodes are merged; past it, the search gives up, *e* is tried, and otherwise the report stands. An inserted terminal is a neighbour token with its terminal and an empty span. For tokens whose text a decoder reads, *repair token f* supplies them, with $f :: 't \Rightarrow$ *span* $\Rightarrow$ *token*. *repair avoid A B* ranks repairs that insert *A* or *B* lower among those that parse equally far (%*avoid-insert*). Under layout the terminal takes the column of the next token if it fits, otherwise the nearest that does.

parser (*diagnostics*) also defines *p-coverage*, the tables and the driver as numbers. An *errors* block is checked for coverage with the method of Pottier (Reachability and error diagnosis in LR(1) parsers, CC 2016): every pair of a state and a terminal that has no action and that some word reaches, each with a shortest word. A pair is covered by a key *PState* of its state or *PToken* of the pair; (*shift*) keys do not count. The states left over come as a warning, with a skeleton of an *errors* block that can be pasted. *parser-error-states p* lists all pairs and the skeleton, and *parser-error-states* (*token*) *p* keys it by pair. With *recover e* no word contains *e*. Each sentence is run on the driver.

On a grammar with columns the automaton without them gives an upper bound, and a pair is listed when a run reaches it, in the default layout (one token after the other) or in another order of the columns. Candidates no run reaches are named, and the list is then not known to be complete; the sentences are shortest for the automaton only. None of this is proved. [[*parser-errors-budget* = *n*]] bounds it to *n* seconds (default *0*, which turns it off; a block is still evaluated for its keys). A coverage pass is a deliberate step, e.g. [[*parser-errors-budget* = *600*]].

on-error-reduce A B (Pottier, Sect. 4) changes what *p-diag* reports: on a rejected word a second run reduces a production of *A* or *B* in any state that has no action on the next terminal, so the error is met in the state after them, which is the one the keys of a block and the listing mean. The parser and its tables are untouched, and *p-diag.Inr-iff* still holds. *p-repair* plans on the parser's own tables and takes its reports from the second run. A state with two such productions takes the lowest one, with a warning; on the simple driver *on-error-reduce* is a warning and is ignored. The second run costs tables of its own in *p-diag*, built when an exported parser is loaded.

token-info f, for tokens with further record fields: *f t* is the value of the

extension (*r.fields* ... for a record *r*) for a token of class *t*. Sentences of symbols and the runs of coverage need it; without it the sentences of coverage are not run, and pairs that depend on columns are left out with a warning.

No theorem is about the *Inl* side, by design: *Inl* - means failure, and only accepted trees matter. *p-diag.Inr-iff* is what makes this safe. Whatever the error code does, it decides nothing, so a report can be wrong but no word changes its status. Recovery is built the same way: it rewrites the word, unverified, and the verified *p* parses the result. Coverage, *on-error-reduce*, *recover* and *repair* are described above; none of them changes *p* or its tables, and all of them run only after a failure, so *p* costs the same.

8 Backend: the LR parser

The only generator at present is *LR* (`lr.ML`). The definition of *p* runs *dparse* on the grammar; its code equations do not.

For code generation the LR tables are constants *p-states*, *p-gtab*, *p-redtab*, *p-i0* and, for the driver, *p-stab*, *p-vtab*, *p-vktab*, *p-sinit* and *p-sclos* (the state-indexed tables of `../lr/LR_Vec.thy`, the sparse closures of `../lr/LR_Sparse.thy`; *p-xinit* and *p-xclos* for a grammar with an ε-cycle), each defined by its term over the ones before it (*p-gtab* = *la-gtab G p-states*); generated code computes each once when it loads. The code equation of *p* refers to those constants, so neither *eval* nor exported code rebuilds the automaton per call. With **declare** [[*parser-literal-tables*]] the six constants are instead the evaluated tables, for exported code that must not compute them at startup; the command then evaluates the tables as well as its checks, including, for the state-indexed driver, *p-stab*, *p-vtab*, *p-vktab*, *p-sinit* and *p-sclos*. If the grammar has no layout annotations, the static determinism check is empty and no lookahead cell of a reduce constrains the width of its token, the code equation instead runs the simple driver of `../lr/LR_Simple.thy`, using the precise LR(0) automaton as well (tables *p-psts*, *p-pgt*, *p-pcomps*, *p-pfins*, *p-p0*). It keeps a stack of state indices and computes no closures at run time; *dparse-eq-sparse* proves that it computes the same result as *dparse*. Its code equation runs on indexed tables, computed from *p-ltabs*: productions and nonterminals are numbered, goto rows are keyed by terminal or by nonterminal number, and the reduces licensed at a state are tabulated by lookahead (*isctl-eq*). These tables are then stored as binary tries keyed by the bits of the index (`../lr/LR_Trie.thy`, *jctl-eq*): they are static data, and with binary naturals a lookup involves no arithmetic, once the code equations *tlookup-binary* are declared next to `Code_Binary_Nat`. The input word is numbered once, pairing each token with its number in the tables (`../lr/LR_Tok.thy`, *xrun-jctl2*). Finally, the

reachable pairs of states of the two automata are numbered, *p-kpairs*, and the tables of the code, *p-ktabs*, are those of this product automaton, with one number per stack entry and the candidate reduces tabulated by lookahead (`../lr/LR_Prod.thy`, *xrun-kctl*); the closure of *p-kpairs* under shifts and gotos is checked by evaluation (*kok*). If the check fails, the code runs on the tries *p-jtabs*. On the product automaton the run detects loops of reductions by comparing with one saved stack instead of all stacks since the last shift (`../lr/LR_Loop.thy`, *xrun-hist-brun*), and a step looks up one action table, *p-ptabs*, by pair number and lookahead: the shift target, or else the reduces with their productions (`../lr/LR_Plain.thy`, *pctl-eq*). If the token type is a datatype whose constructors have no arguments, the number of a token is computed by a case distinction, *p-tnum*, instead of a search in the list of terminals; *pnum p-ptabs* = *p-tnum* is proved constructor by constructor. With **declare** [[*parser-int-tables*]] the tables are instead arrays with integer indices, *p-ztabs*, computed when the code is loaded (`../lr/LR_Int.thy`, *zrun-eq*). For OCaml, `../lr/LR_Simple.thy` implements immutable arrays (`iarray`) by OCaml arrays.

9 Example

A statement is an identifier or a *while* header followed by a block indented further than the header, all statements of a block aligned. Values count statements:

```
datatype tok = While | Colon | Id

grammar (relation) py =
  tokens ‹tok›
  terminals while = ‹While› and : = ‹Colon›
  productions
      Prog :: ‹nat› = Block
    and
      Block :: ‹nat›
        = Stmt=
        | Block= Stmt=  ⇒ ‹(+)›
    and
      Stmt :: ‹nat›
        = Id=                    ⇒ ‹λ-. 1›
        | while= Id> :> Block>   ⇒ ‹λ- n. n + 1›

parser py-parse for py
```

The token class *Id* is a value symbol, so the first action of *Stmt* takes the token record and ignores it. *Block* without annotation leaves the column of the outermost block open, *Block*$^{>}$ after a header demands a deeper one, and *Block*$^{=}$ *Stmt*$^{=}$ aligns the members.

The key theorem generated is

$$py.lang\text{: } (\exists\, x.\ py\text{-}Prog\ 0\ w\ x) = (w \in ilang\ py)$$

where *py-Prog* is the mutual **inductive** relation defined. For sake of brevity we omit the full *py.inducts*/*py.intros* and only show the intros for *py-Stmt*, writing $\langle t;\ sp\rangle$ for the token record $(\!|tok = t,\ span = sp|\!)$ (extension fields elided):

$$\frac{s\text{-}col\ sp = i}{py\text{-}Stmt\ i\ [\langle Id;\ sp\rangle]\ 1}$$

$$\frac{py\text{-}Block\ j\ w\ v \qquad s\text{-}col\ sp_1 = i \qquad i < s\text{-}col\ sp_2 \qquad i < s\text{-}col\ sp_3 \qquad i < j}{py\text{-}Stmt\ i\ (\langle While;\ sp_1\rangle\ \#\ \langle Id;\ sp_2\rangle\ \#\ \langle Colon;\ sp_3\rangle\ \#\ w)\ (v + 1)}$$

The static check is empty for *py*, so completeness is unconditional:

$$py\text{-}parse.complete\text{: } py\text{-}Prog\ 0\ w\ x \Longrightarrow py\text{-}parse\ w = Some\ x$$

With *tu c t* a token of class *t* at column *c*, the word *while x*: at column 1, *y* and *z* at column 3 and *w* at column 1 parses to *Some 4*, while moving *z* to column 4 gives *None*:

lemma ‹*py-parse* [*tu 1 While*, *tu 7 Id*, *tu 8 Colon*, *tu 3 Id*, *tu 3 Id*, *tu 1 Id*] = *Some 4*›
by *eval*

lemma ‹*py-parse* [*tu 1 While*, *tu 7 Id*, *tu 8 Colon*, *tu 3 Id*, *tu 4 Id*] = *None*›
by *eval*

Further examples can be found in `../tests/Regression_Tests.thy`.

References

[1] M. D. Adams. Principled parsing for indentation-sensitive languages: Revisiting Landin's offside rule. In *Proceedings of the 40th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2013)*, pages 511–522. ACM, 2013.